CONST AND CONSTEXPR

Locking values down

CONST

Set once, never reassigned

```
const double taxRate{0.08};
const int categoryCount{4}; // Food, Transport, Entertainment, Other

// taxRate = 0.10; // compiler error: can't reassign a const variable
```

Some values should never change once set - `const` says "initialize once, then never reassign", enforced by the compiler.

MIXING CONST AND NON-CONST

A const result from non-const ingredients

```
double amountSpent{93.33};
const double amountWithTax{amountSpent * (1.0 + taxRate)};
```

A regular variable can still be used to compute a `const` one - the *result* is locked in, even though one of its ingredients wasn't `const`.

CONSTEXPR

Known before the program even runs

```
constexpr double receiptRequiredAbove{25.0};
```

`constexpr` goes a step further than `const`: the value must be computable at **compile time**, before `main()` even starts - not just "never reassigned". Stronger than most everyday values need, but exactly right for a fixed limit like a receipt threshold.

Only what the compiler already knows

```
constexpr int maxCategoriesSupported{8};

// constexpr double badExample{amountSpent}; // compiler error:
// amountSpent is only known once the program is running

constexpr int maxCategoriesDoubled{maxCategoriesSupported * 2}; // fine
```

A constexpr variable can only be initialized from other constexpr values or literals - the compiler works out the whole chain during compilation.

Choosing between them

	WHEN THE VALUE IS KNOWN	CAN IT BE COMPUTED AT COMPILE TIME?
`const`	Once, at runtime (e.g. user input)	Not required
`constexpr`	Before the program runs	Required

DEBUG IT

Constants still show up in locals

Breakpoint after `amountWithTax` is computed - `const` / `constexpr` variables still appear in your IDE's locals view like any other variable, just without an option to edit their value while paused.